Week 5 : Assignment 5

The due date for submitting this assignment has passed.

Due on 2025-08-27, 23:59 IST.

As per our records you have not submitted this assignment.

In school mathematics, we're used to writing equations like:

f(x) = x + 2

This means: "Give me a number x, and I'll give you back x + 2."

Python functions work **exactly like this**, but they need a little more structure so the computer understands.
Let's explore how a machine (Python) thinks about functions

```python
def f(x):
    return x + 2
```

She says:
"This is how you tell the machine: when I give you a number x, return x + 2."

- def means "define a function"
- f(x) means the function's name is f, and it needs one input
- return tells Python what to give back.

So, calling: f(5) will return 7.

This is just like math! But now, you can teach a machine to remember this rule.

More Rules: Multiple Inputs

What if you want to define a function like:

g(x, y) = x * y + 3

In Python:

```python
def g(x, y):
    return x * y + 3
```

Now, you can ask:

g(2, 4)

The function can have **multiple inputs**, and it still returns one output.

Making Reusable Tools

Instead of writing the same calculation again and again, students build small functions like:

```python
def square(n):
  return n * n
```

Or

```python
def is_even(num):
  return num % 2 == 0
```

Bonus Tools: Methods vs Attributes

The PyCalc is upgraded with a NumPy brain to handle data tables (arrays).

The teacher loads:

```python
import numpy as np
data = np.array([[1, 2], [3, 4]])
```

This looks like a matrix or table from math class.

To find out its shape:

```python
print(data.shape)       # prints (2, 2)
```

Here, .shape is an attribute — a fact about the object.

But to reshape it into a new form:

```python
reshaped = data.reshape((4, 1))
```

.reshape() is a method — a function attached to an object, which performs an action.
How to tell the difference?

- No () → attribute → describes something
- With () → method → does something

1) *1 point*

```python
def parity(n):
    return n % 2
```

Which of the following options correctly represents the possible outputs of the function defined above ?

- 0,1
- even, odd
- 5,6
- 2,1

No, the answer is incorrect.
Score: 0
Accepted Answers:
0,1

2) Consider: *1 point*

```python
def f(x):
   return x * 3 + 1
```

What is the result of:

```python
print(f(1), f(2), f(3))
```

- 4 7 10
- 4 6 8
- 3 6 9
- 1 2 3

No, the answer is incorrect.
Score: 0
Accepted Answers:
4 7 10

3) Observe the following: *1 point*

```python
def mystery_function(x):
  return 5
```

What **type of mathematical function** does mystery_function represent?

- **One-to-one**, since different inputs give different outputs
- **Onto**, because all values in the range are covered
- **Many-to-one**, since multiple inputs map to the same output
- **Bijective**, because the function is both one-to-one and onto

No, the answer is incorrect.
Score: 0
Accepted Answers:
***Many-to-one**, since multiple inputs map to the same output*

4) A student defines the following function to simulate a formula from math class: *1 point*

```python
def weighted_sum(x, y=2, z=3):
    return x + y * z
```

They then execute:

```python
result = weighted_sum(1, z=4)
print(result)
```

What will be the output?

- 9
- 21
- 11
- Error: positional argument follows keyword argument

No, the answer is incorrect.
Score: 0
Accepted Answers:
9

5) In the PyCalc system, two functions are defined: *1 point*

```python
def double(x):
    return x * 2

def add_three(x):
    return x + 3
```

A student composes them like this:

```python
def composed(x):
    return add_three(double(x))
```

What is the output of composed(4)?

- 11
- 14
- 7
- 24

No, the answer is incorrect.
Score: 0
Accepted Answers:
11

6) The PyCalc team loaded a NumPy array and wrote the following code: *1 point*

```python
import numpy as np
data = np.array([[10, 20], [30, 40]])
```

A student is asked to print the shape of the array and reshape it into 4 rows and 1 column. They write:

```python
print(data.shape())
reshaped = data.reshape(4, 1)
print(reshaped.shape)
```

What will happen when this code is executed?

- It prints (2, 2) and (4, 1)
- It raises an error on line 1 because reshape cannot be used this way
- It raises an error because .shape is not a function and cannot be called with ()
- It raises an error because reshaping changes the original array

No, the answer is incorrect.
Score: 0
Accepted Answers:
It raises an error because .shape is not a function and cannot be called with ()

7) *1 point*

```python
def manual_sum(lst):
    total = 0
    for i in lst:
        total = total + i
    return total

def process_list(L):
    squares = []
    for item in L:
        squares.append(item ** 2)
    return squares, manual_sum(squares)

result = process_list([1, 2, 3])
print(result[0][1], result[1])
```

What will be printed?

- 2 6
- 1 6
- 4 14
- 9 6

No, the answer is incorrect.
Score: 0
Accepted Answers:
4 14

8) *1 point*

```python
def identity():
  return "Function called"
identity_value = "Just a value"
```

Later, they write the following code:

```python
print(identity)
print(identity())
print(identity_value())
```

What will happen when this code is run?

- Function called
  Function called
  Just a value
- <function identity at ...>
  Function called
  Error: 'str' object is not callable
- <function identity at ...>
  <function identity at ...>
  Just a value
- Function called
  Error
  Error

No, the answer is incorrect.
Score: 0
Accepted Answers:
<function identity at ...>
Function called
Error: 'str' object is not callable

9) A student is trying to understand the difference between variables and functions in Python. They write the following: *1 point*

```python
def message():
    return "This is a function"

message = "This is a variable"

print(message())
```

What will happen when this code runs?

- This is a function
- This is a variable
- Error: 'str' object is not callable
- None

No, the answer is incorrect.
Score: 0
Accepted Answers:
Error: 'str' object is not callable

10) A student defines the following function to collect error logs during program execution: *1 point*

```python
def log_error(msg, log=[]):
    log.append(msg)
    return log
```

They run the following code:

```python
print(log_error("E1"))
print(log_error("E2"))
print(log_error("E3", []))
```

What will be the output?

- ['E1']
  ['E2']
  ['E3']
- ['E1']
  ['E1', 'E2']
  ['E3']
- ['E1']
  ['E1']
  ['E3']
- ['E1']
  ['E2']
  ['E3', 'E1']

No, the answer is incorrect.
Score: 0
Accepted Answers:
['E1']
['E1', 'E2']
['E3']

In computer science, searching algorithms are fundamental techniques used to locate specific data within a collection, such as an array or list.

Linear Search is the simplest searching algorithm. It works by checking each element one by one, starting from the beginning of the list, until the desired element is found or the end of the list is reached. It does not require the list to be sorted.

Here is a Python implementation of linear search:

```python
def linear_search(arr, target):
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
```

In contrast, **Binary Search** is a more efficient algorithm, but it only works on **sorted arrays**. It repeatedly divides the search interval in half. If the target value is less than the element in the middle, the search continues in the left half. Otherwise, it continues in the right half. This continues until the target is found or the interval is empty.

Here is a Python implementation of binary search:

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

11) What is the time complexity of the worst-case scenario for linear search? *1 point*

- O(1)
- O(log n)
- O(n)
- O(n log n)

No, the answer is incorrect.
Score: 0
Accepted Answers:
O(n)

12) Which condition must be met for binary search to function correctly? *1 point*

- The array must have unique elements
- The array must be unsorted
- The array must be sorted
- The array size must be even

No, the answer is incorrect.
Score: 0
Accepted Answers:
The array must be sorted

13) What will be the output of binary_search([1, 3, 5, 7, 9], 7)? *1 point*

- 2
- 3
- 4
- 1

No, the answer is incorrect.
Score: 0
Accepted Answers:
3

14) In the binary search algorithm, what happens if arr[mid] > target? *1 point*

- Search left half of the array
- Search right half of the array
- Return mid
- Return -1

No, the answer is incorrect.
Score: 0
Accepted Answers:
Search left half of the array

15) In the following code, what value is returned? *1 point*

```python
def linear_search(arr, target):
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
arr = [10, 20, 30, 40]
print(linear_search(arr, 25))
```

- 2
- 3
- -1
- Error

No, the answer is incorrect.
Score: 0
Accepted Answers:
-1

16) For a sorted list of 1,000,000 elements, which search algorithm is generally faster for finding a value in terms of average-case performance? *1 point*

- Linear Search
- Binary Search
- Both are equally fast
- Performance depends mainly on the programming language

No, the answer is incorrect.
Score: 0
Accepted Answers:
Binary Search

17) Using the binary_search implementation above, what is the maximum number of times the comparison arr[mid] == target can be executed when searching a sorted list of 256 elements? *1 point*

- 256
- 128
- 8
- 9

No, the answer is incorrect.
Score: 0
Accepted Answers:
9

18) Which statement is FALSE? *1 point*

- Linear search does not need sorted data
- Binary search needs sorted data
- Binary search always performs better than linear search
- Linear search checks every element until it finds the match

No, the answer is incorrect.
Score: 0
Accepted Answers:
Binary search always performs better than linear search

19) *1 point*

```python
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1
arr = [3, 6, 9, 12, 15]
print(binary_search(arr, 4))
```

What will be the output of the following code?

- 0
- -1
- 1
- None

No, the answer is incorrect.
Score: 0
Accepted Answers:
-1

20) If the element you are searching for is known to be at the end of a 100-element list and you use linear search, how many comparisons will it take? *1 point*

- 100
- 50
- 25
- 10

No, the answer is incorrect.
Score: 0
Accepted Answers:
100

The **Monty Hall Problem** is a famous probability puzzle based on a game show scenario. It presents a counterintuitive result that surprises many people when they first encounter it.

The Problem Setup:

You are a contestant on a game show. There are **three doors:**

- Behind **one door** is a **car** (the prize).
- Behind the other **two doors** are **goats**.

The game proceeds as follows:
1. You choose one of the three doors.
2. The host (Monty Hall), who **knows what's behind the doors**, opens one of the **other two doors** to reveal a **goat**.
3. You are then given a choice: **stick** with your original choice, or **switch** to the remaining unopened door.

Should You Switch or Stay?

Surprisingly, **switching gives you a higher chance of winning** the car.

- If you **stick**: your chance of winning = **1/3**
- If you **switch**: your chance of winning = **2/3**

This result can be verified through **simulation** using lists in Python.

```python
import random

def simulate_monty_hall(switch: bool) -> bool:
    doors = ['goat', 'goat', 'car']
    random.shuffle(doors)  # Randomize door positions

    # Contestant makes a random choice
    choice = random.randint(0, 2)

    # Host opens a door with a goat (not the chosen one)
    available_doors = [i for i in range(3) if i != choice and doors[i] == 'goat']
    monty_opens = random.choice(available_doors)

    # If switch is True, change the choice to the remaining unopened door
    if switch:
        remaining_doors = [i for i in range(3) if i != choice and i != monty_opens]
        choice = remaining_doors[0]

    return doors[choice] == 'car'
```

You can simulate this many times to check the **probability of winning** by switching vs. staying:

```python
trials = 10000
switch_wins = sum(simulate_monty_hall(True) for _ in range(trials))
stay_wins = sum(simulate_monty_hall(False) for _ in range(trials))

print(f"Win percentage when switching: {switch_wins / trials * 100:.2f}%")
print(f"Win percentage when staying: {stay_wins / trials * 100:.2f}%")
```

21) In the Monty Hall simulation, why does switching give a higher probability of winning the car, even though there are only two doors left at the end? *1 point*

- The host reveals the car
- The initial choice had a 1/3 chance of being correct, so switching captures the remaining 2/3
- The remaining door is chosen randomly
- All options become equally likely after Monty's reveal

No, the answer is incorrect.
Score: 0
Accepted Answers:
The initial choice had a 1/3 chance of being correct, so switching captures the remaining 2/3

22) In the following Python code, what does the line marked with # ensure ? *1 point*

```python
available_doors = [i for i in range(3) if i != choice and doors[i] == 'goat']  #
monty_opens = random.choice(available_doors)
```

- It guarantees the host never reveals the car
- It helps the contestant win more often
- It randomly opens a car door
- It changes the position of the car

No, the answer is incorrect.
Score: 0
Accepted Answers:
It guarantees the host never reveals the car

23) What would happen if the list available_doors included doors that might have the car behind them? *1 point*

- The switch strategy would always win
- The simulation would stop with an error
- The probability advantage of switching would no longer hold
- It would make the list empty

No, the answer is incorrect.
Score: 0
Accepted Answers:
The probability advantage of switching would no longer hold

24) What is the main purpose of random.shuffle(doors) in the simulation? *1 point*

- To select a goat door
- To ensure fairness by randomly placing the car
- To ensure Monty opens the correct door
- To make the output deterministic

No, the answer is incorrect.
Score: 0
Accepted Answers:
To ensure fairness by randomly placing the car

25) Which of the following modifications would incorrectly simulate the Monty Hall problem? *1 point*

- Removing the use of random.shuffle() when placing the car behind one of the doors.
- Allowing Monty to randomly open any unchosen door, even if it hides the car.
- Always forcing the player to switch their initial choice after Monty opens a door.
- Running the simulation 10,000 times to observe overall win rates.

No, the answer is incorrect.
Score: 0
Accepted Answers:
Allowing Monty to randomly open any unchosen door, even if it hides the car.

26) What does this snippet do in the simulation? *1 point*

```python
remaining_doors = [i for i in range(3) if i != choice and i != monty_opens]
choice = remaining_doors[0]
```

- Selects the door with the car
- Randomly opens a door
- Switches the player's choice to the remaining unopened door
- Forces the player to lose

No, the answer is incorrect.
Score: 0
Accepted Answers:
Switches the player's choice to the remaining unopened door

27) Why is running the simulation with thousands of trials (e.g., 10,000) important? *1 point*

- To stress test the program
- To ensure the code doesn't crash
- To approximate the actual win probabilities using empirical evidence
- Because smaller numbers always produce 100% win rates

No, the answer is incorrect.
Score: 0
Accepted Answers:
To approximate the actual win probabilities using empirical evidence

28) You pick one of the three doors at random. What is the probability that your initial choice is **wrong**? *1 point*

- 1/3
- 1/2
- 2/3
- Cannot be determined

No, the answer is incorrect.
Score: 0
Accepted Answers:
2/3

29) You pick Door 1. Monty, knowing the prize location, opens Door 3, which has a goat. What is the probability the car is behind Door 2 if you initially picked a goat? *1 point*

- 1/3
- 1/2
- 1
- 2/3

No, the answer is incorrect.
Score: 0
Accepted Answers:
1

30) What kind of probability concept does the Monty Hall problem mainly help us understand? *1 point*

- How to calculate the total number of possible events
- How to identify equally likely outcomes
- How probability changes when new information is revealed
- How to simulate random experiments using coins and dice

No, the answer is incorrect.
Score: 0
Accepted Answers:
How probability changes when new information is revealed